

Amazon Internet?

Amazon may launch an internet service, which might be bundled along with Prime. <http://dgit.in/AmazInt>



Next-gen Chess board

Square Off chessboard can move pieces around with no touching involved. <http://dgit.in/SOChsBrd>

character will now collect the gem. With this, you are done with the first lesson. Select next to start with the second.

This time, there's a new command called "turnLeft()". This means you need to first figure out how many tiles there are between the Byte and the gem every single time. In this lesson, there are four. So the "moveForward()" command should be used four times. However, this lesson demands you turn the character to the left. So use the "moveForward()" command two times followed by "turnLeft()". Again use "moveForward()" twice and finally "collectGem()". Hit the 'run' button once you are done.

The third lesson adds a new command "toggleSwitch()", which toggles the switch as open or closed. Analyse how you'll have to write the code. There are three tiles between the Byte and the gem, and three between the gem and the switch. You'll have to make Byte turn and also use the 'collect gem' command in the interim period. Run the code after you are done.

The fourth lesson is even more interesting. Now you have to teleport Byte to collect the gem. Analyse the map and see what steps you have to take and apply them. You may refer to the hints in the bottom right corner for help.

The fifth lesson is all about "Finding and fixing bugs". Here, you are already provided with the list of commands. Their order, however, is changed. If you run your code, Byte won't be able to collect the gem because there is a bug (mistake) in the program. Carefully analyse the map and rearrange the existing list of commands to perform the right actions.

The sixth lesson is again about finding and fixing bugs. Here, if you analyse the commands they are all messed up and if you run the code, you'll see Byte taking the wrong steps. Here you'll have to make lot of changes to the existing code. Analyse the map, double tap the command you want to rearrange, and drag it to its appropriate position. Do this to fix the code and make Byte collect the gem.

The seventh lesson is a tricky one. Here, you have many ways to collect the

gem, but the most appropriate one is to use the shortest path. The challenge is specially designed in a way so as to make you put all the skills (including debugging) you have learnt to use. The trial and error method should make it work. If you get it wrong undo the command and try a different one. Here are the commands if you are unable to make it work: <http://dgit.in/7thlesson>

The next section "Introduction to functions" will begin after you've completed this challenge. The introduction will give you a lowdown about the functions and their uses. Complete all the sessions in the chapter in a similar



Hit the "+" button to start. All your creations will appear in this area later.

fashion. Chapter 2 will get you onto a more advanced level. With this kind of a step-by-step learning method, you'll be able to grasp the basics of coding in no time.

Playing with Playgrounds

If you have completed the lessons in the Featured area, you can now get your hands dirty and tinker around with your skills. To do this, simply go to the "My Playgrounds" tab and hit the "+" button. You'll be presented with three templates to choose from: Answers, Blank, and Shapes. The first is a Q&A, the second an unrestricted playground that can be used for anything you want to code and the last one lets you play with shapes and its attributes in code.

Answers Template

Here you can pose any question using code and manipulate your answer or

display it. You'll see pre-generated code that asks you for your name. Here's an example:

```
show("What is your name?")
```

The method 'show()' is used to print a text on the output console. Whatever you write within the parenthesis and between the double quotes appears on the screen, which here is "What is your name".

```
let name = ask("Name")
```

The ask() method creates a textbox for you to enter your name. Anything written within the parenthesis and between the double quotes in the method will be displayed as the placeholder in the text box. The ask method returns a string (a set of characters). When you type your name out, it should be stored in some variable, which will let you manipulate it later. In the example above, let name = ask("Name") does the same thing. Whatever you type in the textbox created by the ask method will be stored in the variable called name.

```
Show ("Hi" + name)
```

You see a different format of the show method here. In this case, "Hi" will be printed directly because it is between double quotes. Additionally, there's a plus sign and our variable name. In programming languages, the "+" operator is used to append (join) text. The plus sign is followed by a variable "name". Thus, the show method will display "Hi <your entered name>". This is because the string "Hi" is joined with a variable, which is currently holding data that you entered through the textbox.

Create a pseudo information system

You may use the question-answer template to make a pseudo information database.

The code can look something like the code at this link: <http://dgit.in/QnASWft>

Shapes Template

As the name of the template suggests, you can work on different shapes -